

8.2-Multiple_Linear_Regression

July 24, 2025

0.0.1 Multiple Linear Regression

```
[1]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
%matplotlib inline
import warnings
warnings.filterwarnings('ignore')
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LinearRegression
```

```
[2]: df = pd.read_csv('../0-Dataset/economic_index.csv')
df.head()
```

```
[2]:
```

	Unnamed: 0	year	month	interest_rate	unemployment_rate	index_price
0	0	2017	12	2.75	5.3	1464
1	1	2017	11	2.50	5.3	1394
2	2	2017	10	2.50	5.3	1357
3	3	2017	9	2.50	5.3	1293
4	4	2017	8	2.50	5.4	1256

```
[3]: ### Drop unneccessary columns
df.drop(columns = ['Unnamed: 0', 'year', 'month'], axis=1, inplace=True)
```

```
[4]: df.head()
```

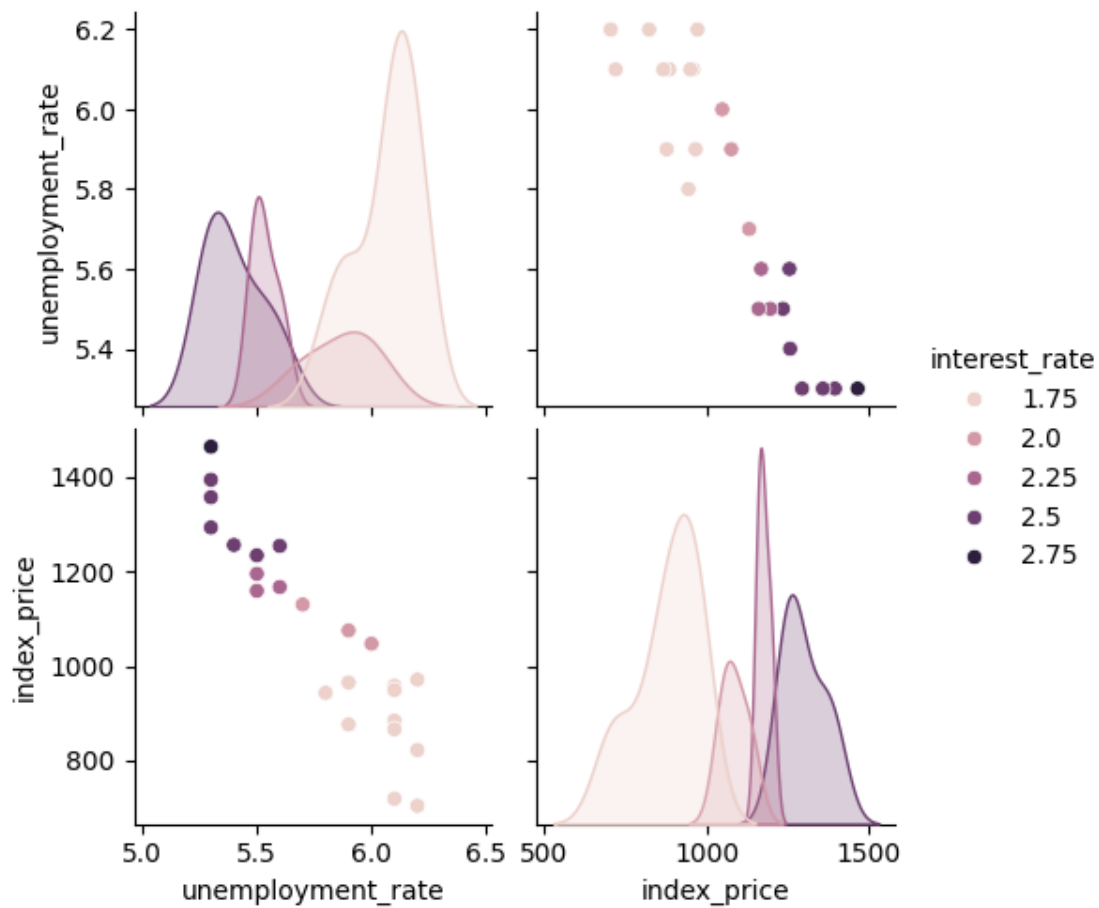
```
[4]:
```

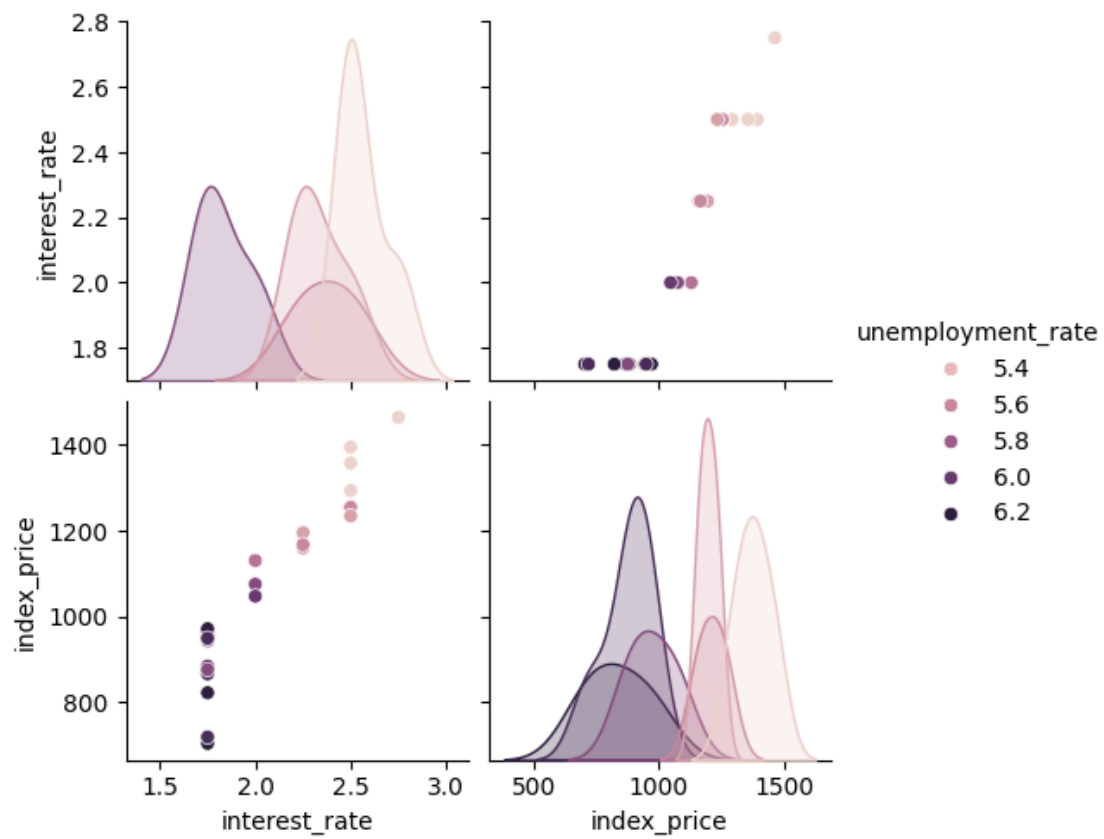
	interest_rate	unemployment_rate	index_price
0	2.75	5.3	1464
1	2.50	5.3	1394
2	2.50	5.3	1357
3	2.50	5.3	1293
4	2.50	5.4	1256

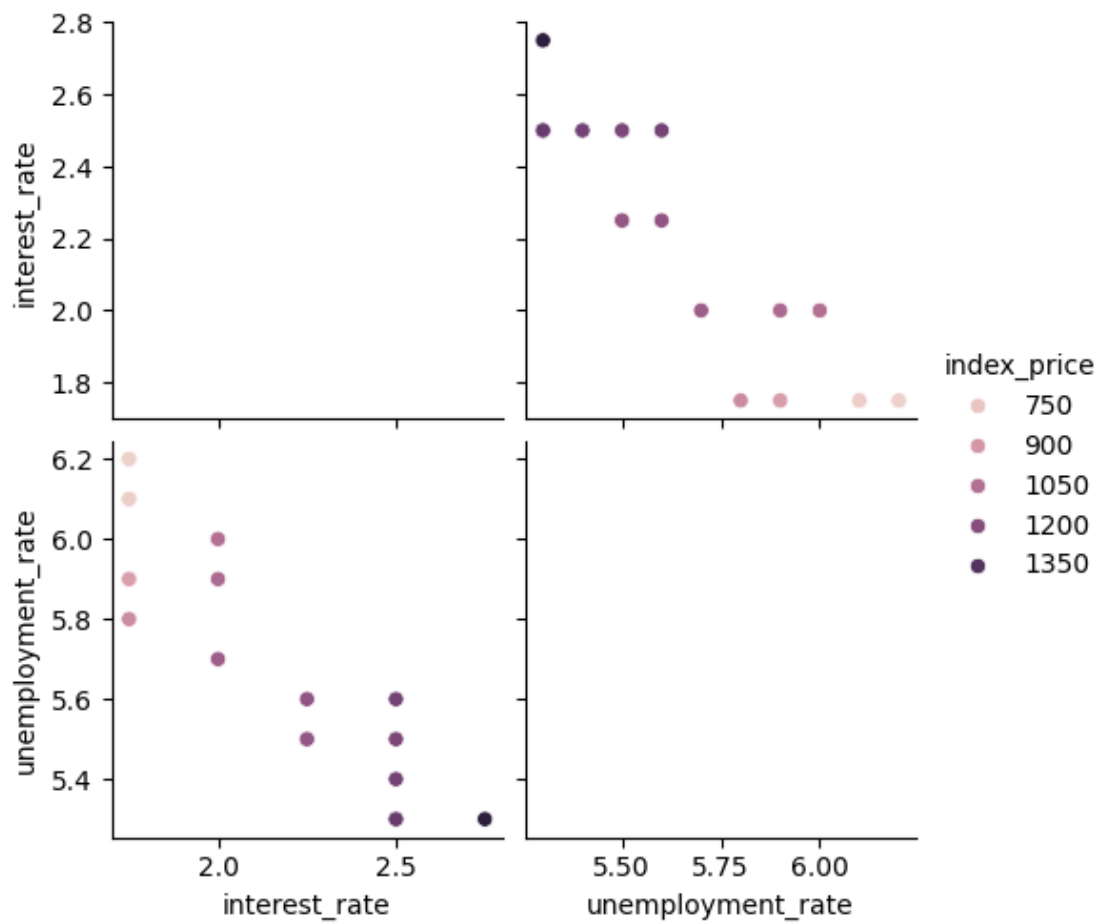
```
[5]: df.isnull().sum()
```

```
[5]: interest_rate      0
      unemployment_rate  0
      index_price       0
      dtype: int64
```

```
[6]: ### Lets do some visualization
      for column in df.columns:
          sns.pairplot(df, hue=column)
```

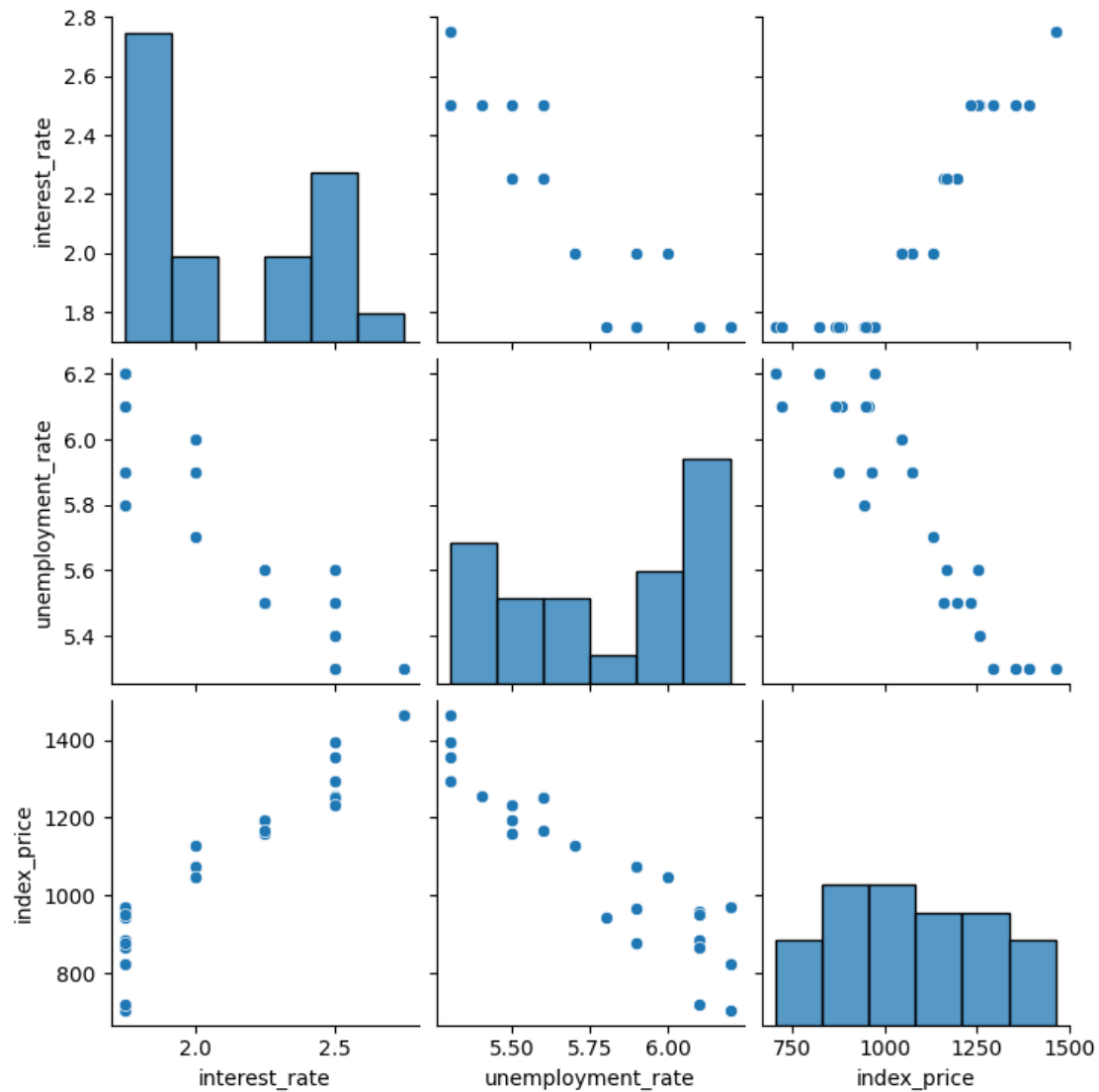






```
[7]: sns.pairplot(df)
```

```
[7]: <seaborn.axisgrid.PairGrid at 0x1cf6c356ce0>
```



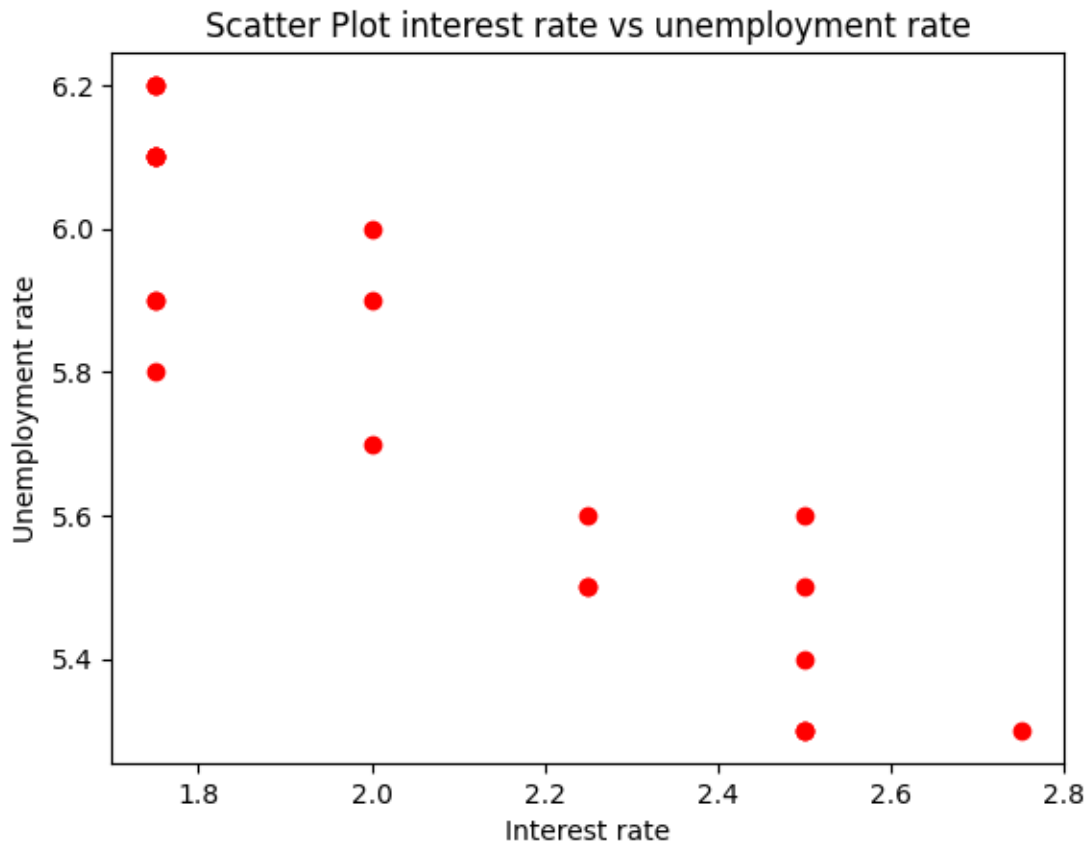
```
[8]: df.corr()
```

```
[8]:
```

	interest_rate	unemployment_rate	index_price
interest_rate	1.000000	-0.925814	0.935793
unemployment_rate	-0.925814	1.000000	-0.922338
index_price	0.935793	-0.922338	1.000000

```
[9]: ### Visualize the datapoint more closely
plt.scatter(df['interest_rate'], df['unemployment_rate'], color='r')
plt.title('Scatter Plot interest rate vs unemployment rate')
plt.xlabel('Interest rate')
plt.ylabel('Unemployment rate')
plt.show()
```

```
plt.close()
```



```
[10]: ### Independent and Dependent features  
X = df.iloc[:, :-1] ## it says that take all the columns except last one  
y = df.iloc[:, -1] ## it says that take only last columns
```

```
[11]: X.head()
```

```
[11]:
```

	interest_rate	unemployment_rate
0	2.75	5.3
1	2.50	5.3
2	2.50	5.3
3	2.50	5.3
4	2.50	5.4

```
[12]: y.head()
```

```
[12]:
```

0	1464
1	1394
2	1357

```
3    1293
4    1256
Name: index_price, dtype: int64
```

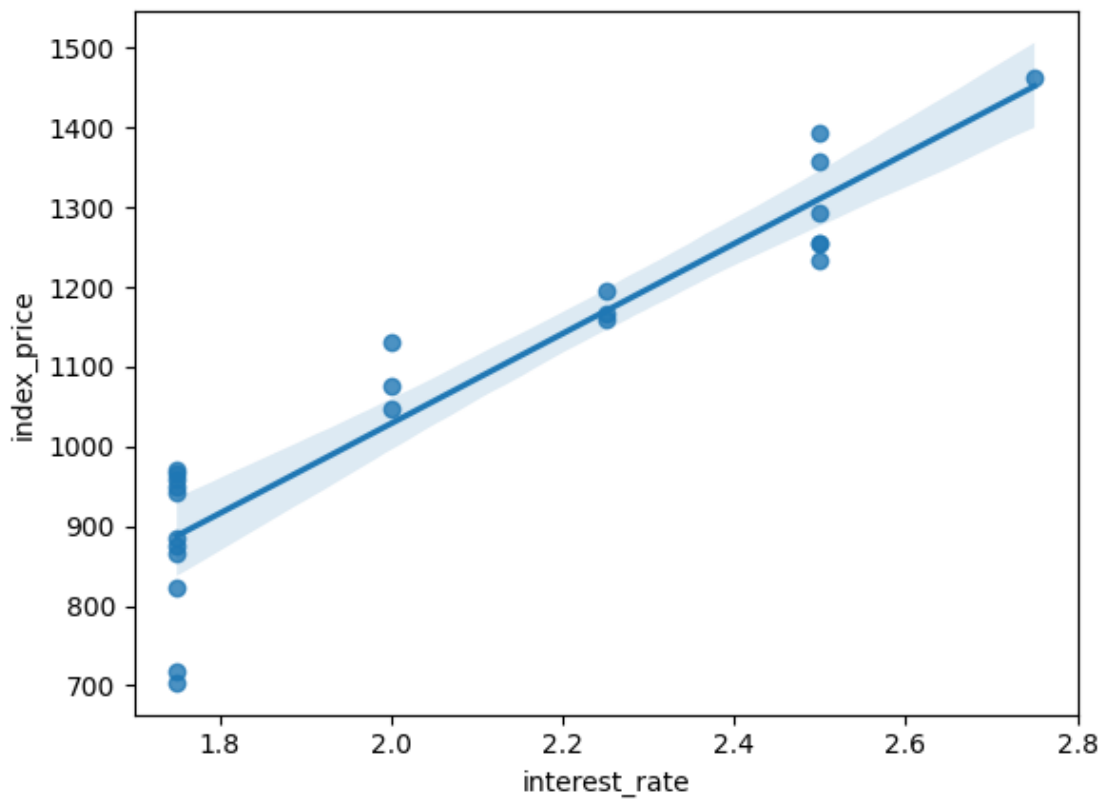
```
[13]: X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.25,
↳ random_state=42)
```

```
[14]: X_test.shape, X_train.shape, y_train.shape, y_test.shape
```

```
[14]: ((6, 2), (18, 2), (18,), (6,))
```

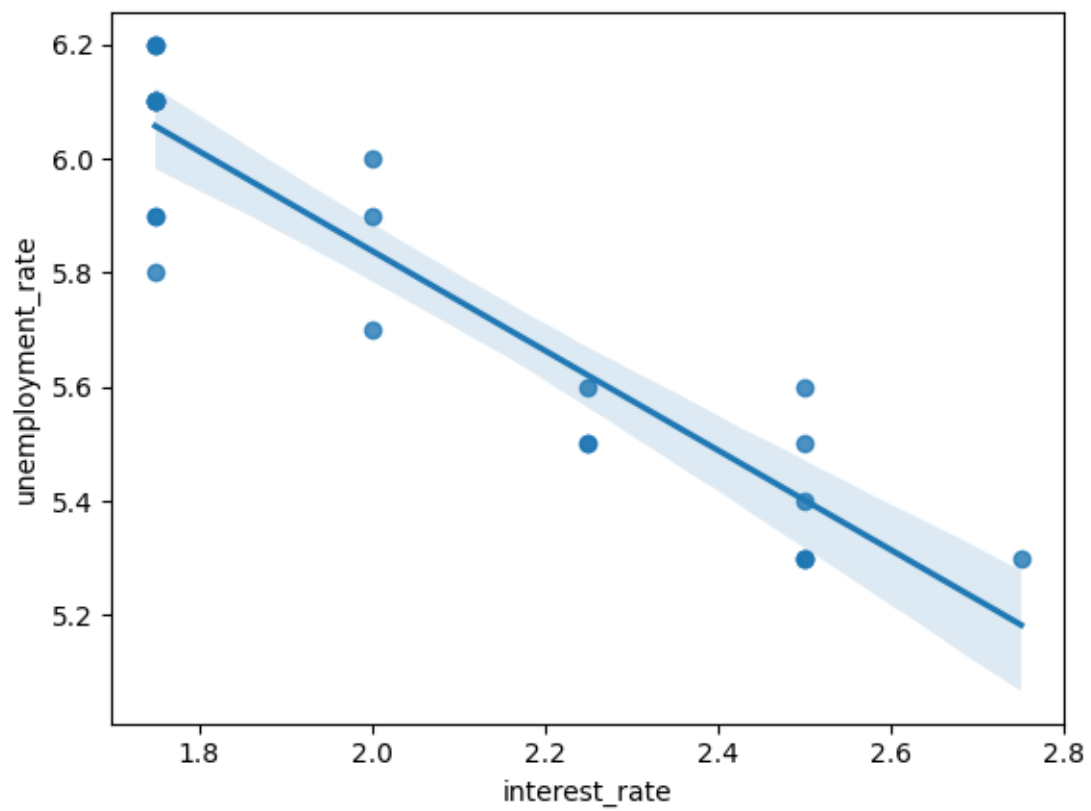
```
[15]: sns.regplot(x='interest_rate', y='index_price', data=df)
```

```
[15]: <Axes: xlabel='interest_rate', ylabel='index_price'>
```



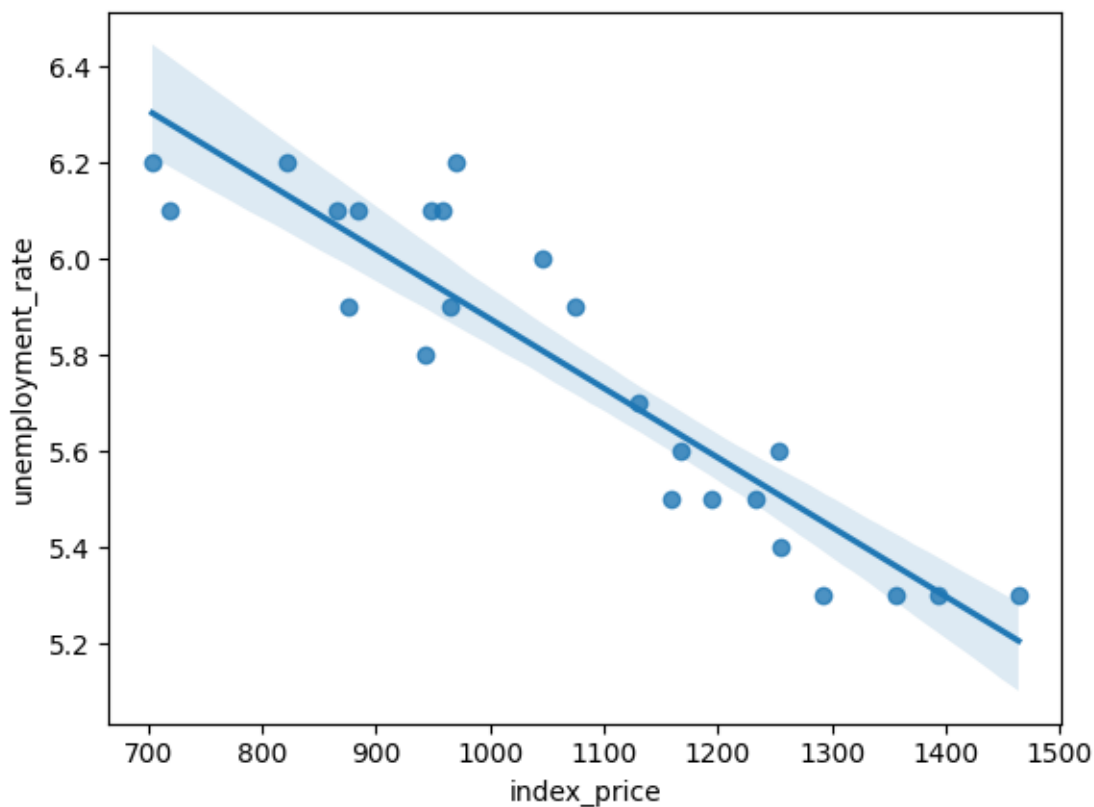
```
[16]: sns.regplot(x='interest_rate', y='unemployment_rate', data=df)
```

```
[16]: <Axes: xlabel='interest_rate', ylabel='unemployment_rate'>
```



```
[17]: sns.regplot(x='index_price', y='unemployment_rate', data=df)
```

```
[17]: <Axes: xlabel='index_price', ylabel='unemployment_rate'>
```



```
[18]: scaler = StandardScaler()
      X_train = scaler.fit_transform(X_train)
      X_test = scaler.transform(X_test)
```

```
[19]: X_train
```

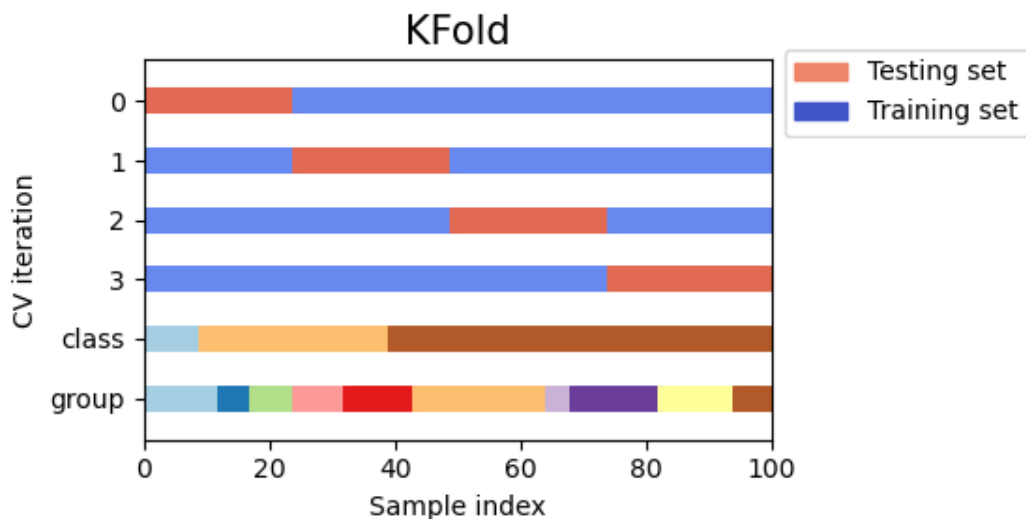
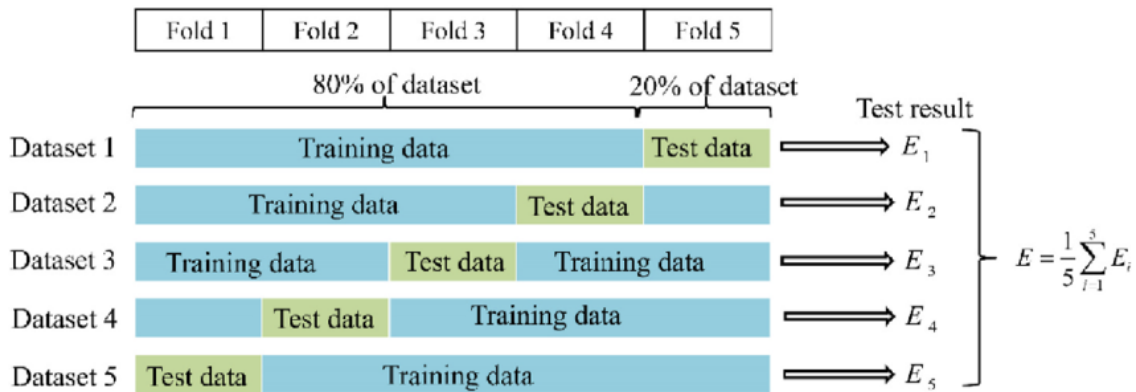
```
[19]: array([[ -0.90115511,  0.37908503],
             [ 1.31077107, -1.48187786],
             [-0.90115511,  1.30956648],
             [ 1.31077107, -0.55139641],
             [ 1.31077107, -1.48187786],
             [-0.16384638,  0.68924552],
             [-0.90115511,  0.999406  ],
             [ 1.31077107, -1.48187786],
             [ 1.31077107, -1.17171738],
             [-0.90115511,  1.30956648],
             [-0.90115511,  0.999406  ],
             [-0.90115511,  0.37908503],
             [-0.90115511,  0.999406  ],
             [ 0.57346234, -0.8615569 ],
             [-0.16384638, -0.24123593],
```

```
[-0.90115511, 0.06892455],
[-0.90115511, 0.999406 ],
[ 1.31077107, -0.8615569 ]])
```

```
[20]: regression = LinearRegression()
      regression.fit(X_train, y_train)
```

```
[20]: LinearRegression()
```

Cross Validation `sklearn.model_selection.cross_validate(estimator, X, y, scoring, cv)` Here when we are going for the cross validation then it is taking the parameters of estimator that is nothing but our model then we give the X and Y values and then for the error calculation we use the `scoring="error-type"` and then use the cross validation for the no of iterations for testing and training the datasets



Iterations:

```
[21]: from sklearn.model_selection import cross_val_score
      validation_score = cross_val_score(regression, X_train, y_train,
      ↪scoring='neg_mean_squared_error', cv=3)
```

```
[22]: validation_score
```

```
[22]: array([-4921.61331265, -7686.87497294, -5135.9962549 ])
```

```
[23]: np.mean(validation_score)
```

```
[23]: -5914.828180162386
```

Prediction

```
[24]: y_pred = regression.predict(X_test)
```

```
[25]: y_pred
```

```
[25]: array([1204.22770398,  821.65051903, 1406.51300368,  857.70889608,  
          994.90992298, 1168.16932693])
```

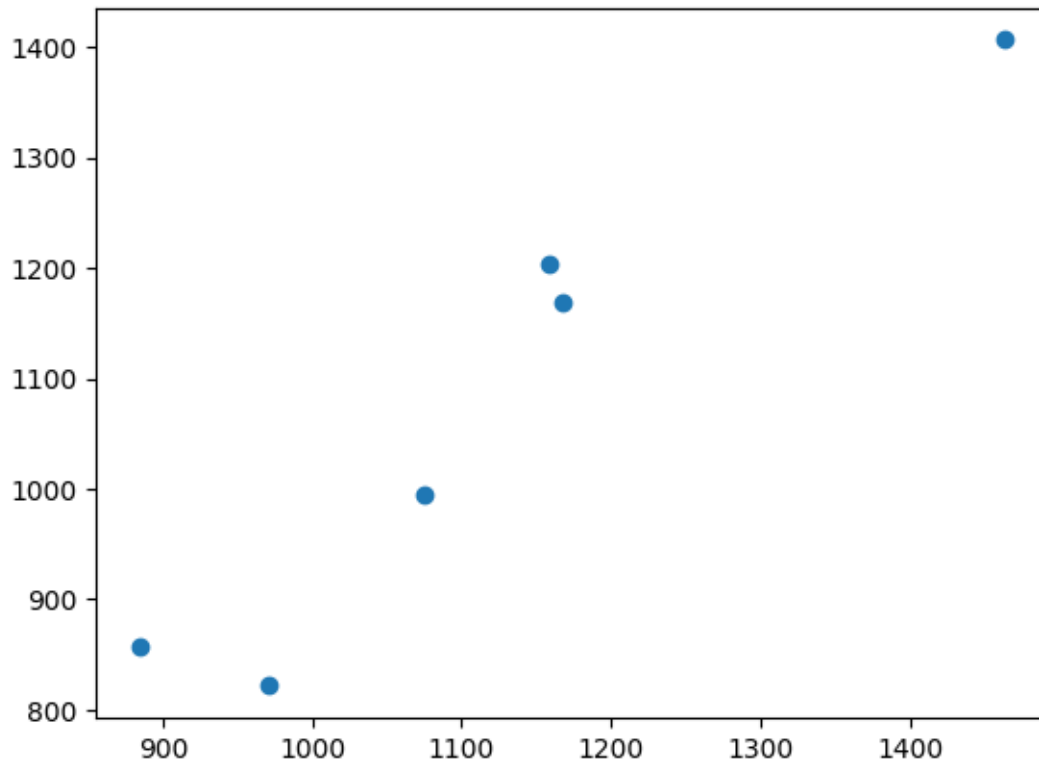
```
[26]: from sklearn.metrics import mean_absolute_error, mean_squared_error  
mse = mean_squared_error(y_test, y_pred)  
mae = mean_absolute_error(y_test, y_pred)  
rmse = np.sqrt(mse)  
print(mse)  
print(mae)  
print(rmse)
```

```
5793.762887712569  
59.935781523235484  
76.11677139574805
```

0.0.2 Assumptions

```
[27]: plt.scatter(y_test, y_pred)
```

```
[27]: <matplotlib.collections.PathCollection at 0x1cf70e3dd20>
```

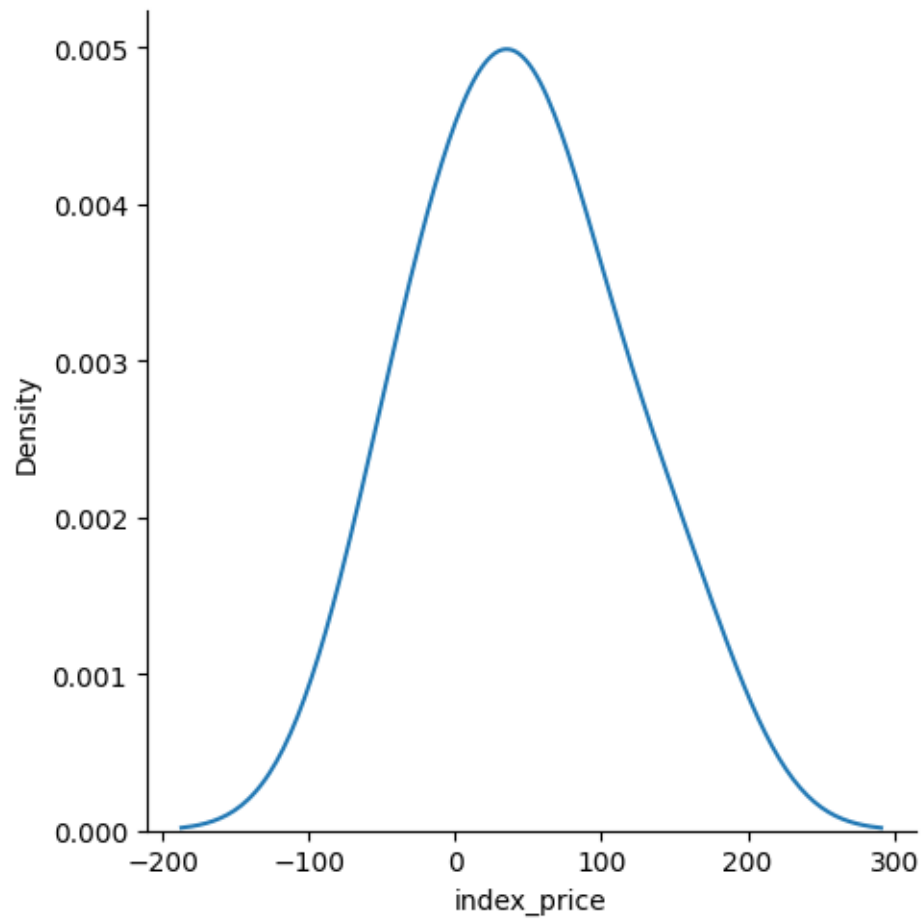


```
[28]: residuals = y_test-y_pred  
residuals
```

```
[28]: 8      -45.227704  
16     149.349481  
0       57.486996  
18      26.291104  
11      80.090077  
9       -1.169327  
Name: index_price, dtype: float64
```

```
[29]: sns.displot(residuals, kind = 'kde')
```

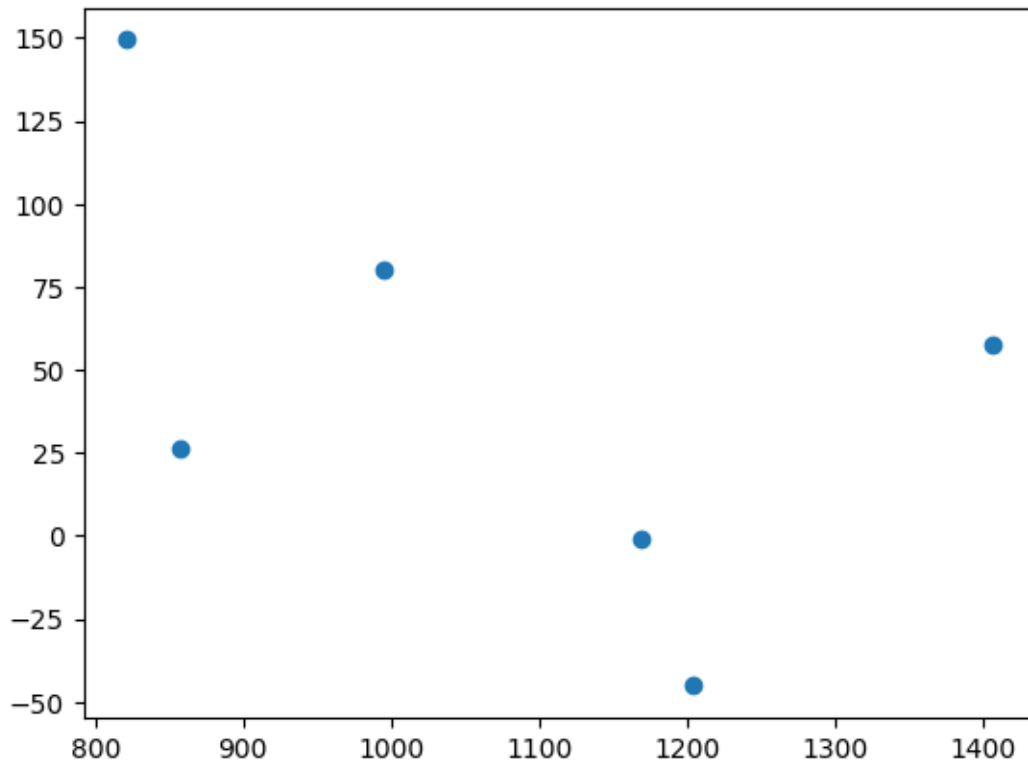
```
[29]: <seaborn.axisgrid.FacetGrid at 0x1cf6e61f460>
```



```
[30]: ### Scatter plot wrt prediction and residuals
```

```
plt.scatter(y_pred, residuals)
```

```
[30]: <matplotlib.collections.PathCollection at 0x1cf709a4130>
```



```
[34]: regression.coef_
```

```
[34]: array([ 88.27275507, -116.25716066])
```

```
[31]: import statsmodels.api as sm
model = sm.OLS(y_train, X_train).fit()
```

```
[33]: print(model.summary())
```

```

                                OLS Regression Results
=====
=====
Dep. Variable:                index_price    R-squared (uncentered):
0.035
Model:                        OLS          Adj. R-squared (uncentered):
-0.086
Method:                       Least Squares    F-statistic:
0.2880
Date:                         Thu, 24 Jul 2025    Prob (F-statistic):
0.754
Time:                         00:27:02          Log-Likelihood:
-150.85
No. Observations:              18              AIC:
```

305.7

Df Residuals: 16 BIC:

307.5

Df Model: 2

Covariance Type: nonrobust

	coef	std err	t	P> t	[0.025	0.975]
x1	88.2728	658.295	0.134	0.895	-1307.250	1483.796
x2	-116.2572	658.295	-0.177	0.862	-1511.780	1279.266
Omnibus:	0.598		Durbin-Watson:		0.007	
Prob(Omnibus):	0.741		Jarque-Bera (JB):		0.567	
Skew:	-0.361		Prob(JB):		0.753	
Kurtosis:	2.517		Cond. No.		4.78	

Notes:

[1] R^2 is computed without centering (uncentered) since the model does not contain a constant.

[2] Standard Errors assume that the covariance matrix of the errors is correctly specified.